SWC3

Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



9. C++ Einführung

»People who think
they know everything
really annoy those of us
who know we don't.«

— *Bjarne Stroustrup*

»C++ has been around forever.
Modern C++ is not your daddy's C++.
We have to help people unlearn
some negative things
that are no longer true
and in some cases never were true.«

— *Herb Sutter*

Geschichte von C++

- ab 1979 “*C with classes*” von Bjarne Stroustrup als Präprozessor für C (`Cpre`)
  - inspiriert durch
    - *BCPL* von Martin Richards (IEEE Computer Pioneer Award 2003)
    - Simula (Klassen, Vererbung) aber sehr langsam
- 1982 erster C++ Compiler: Cfront
- 1985 “The C++ Programming Language” von B. Stroustrup
- 1989 C++ 2.0: Mehrfachvererbung, abstrakte, statische und const Members
- 1998 ISO Standard C++98: templates, exceptions, namespaces, boolean
- 2003 C++03: kleines Updates
- 2011 C++11: Erweiterung Standardbibliothek, Move-Semantik `T&&`, `constexpr`, Typinferenz `auto`, Range-based loops, ...
- 2014 C++14: kleines Update
- 2017 C++17: Compile-time `if`, Destructuring, `std::string_view`, `std::optional`, `std::variant`
- 2020 C++20: Concepts, Modules, `<=>`-Operator, Ranges, `consteval`
- 2023 C++23: `std::print`, `std::expected`, `std::unreachable`, `std::views`, ...

Popularität von C++

Quelle	Platz	Stand
TIOBE	2	Jul 2025
PyPL	3	Aug 2025
RedMonk	7	Jun 2025
IEEE Spectrum	6[1]	Aug 2024
GitHut 2.0	5[1]	Mar 2024

[1] Durchschnitt aus mehreren Reihungen

Eigenschaften von C++ (i)

- imperativ vs. deklarativ
  - direkte Kontrolle über Programmablauf und Änderungen
- prozedural, objektorientiert und funktional
  - Organisation in Prozeduren und Klassen
  - (Mehrfach-)Vererbung
  - abstrakte Klassen als Schnittstellen (*interfaces*)
  - (anonyme) Funktionen als wichtiger Bestandteil
- statisch typisiert vs. dynamisch typisiert
  - Typ einer Variablen ist dem Compiler bekannt
- relativ stark typisiert vs. schwach typisiert
  - Typ einer Variablen kann nicht so leicht verändert bzw. neu interpretiert werden
- übersetzt vs. interpretiert
  - C++ wird von einem Compiler in Maschinensprache übersetzt
  - keine Laufzeitumgebung (*runtime*) oder virtuelle Maschine
  - spürbar längere Übersetzungszeit als C

Eigenschaften von C++ (ii)

- low-level bis high-level
  - C++ deckt breites Spektrum von Hardwarenähe ab
  - teilweise ähnlich zu C
  - teilweise sehr hohe Abstraktionskonzepte
- zero overhead principle
  - kein Performanzverlust für Features die nicht verwendet werden
  - gebotene Features sind genauso effizient wie “von Hand” geschrieben
- manuelle Speicherverwaltung vs. Garbage Collection
  - planbare Performance
  - smart pointer
  - automatische Variablen in modernem C++
  - resource acquisition is initialization (RAII)
- portabel und standardisiert vs. plattformspezifisch
  - leichte Verfügbarkeit auf verschiedenen Systemen

Eigenschaften von C++ (iii)

- umfangreich vs. kompakt
  - große Standardbibliothek
  - viele Programmierparadigmen und -konzepte
- generisch durch Templates (Schablonen)
  - Quelltext kann für verschiedene Datentypen vom Compiler abgeleitet werde
    - Container für verschiedene Datentypen
    - Algorithmen für verschiedene Container
- Überladen von Operatoren
  - z.B. eigene Addition von Bruchzahlen oder Vektordatentyp
- unterstützt Ausnahmen (*exceptions*)
  - Programmfluss kann direkt zur Ausnahmenbehandlung geleitet werden
- Präprozessor
  - Quelltext wird vor der Übersetzung vorverarbeitet
  - erlaubt Textersetzungen (Makros)

Literatur

Hauptquellen

- U. Breymann *“C++ Programmieren”*, 6.ed, Hanser, 2020
- C++ reference - cppreference.com
- C++ Core Guidelines

- ISO/IEC 14882:2024 *“ISO Standard: C++23”* (n4950)
- B. Stroustrup *“A Tour of C++”*, 2.ed, Addison-Wesley, 2018
- A. Koenig & B. Moo *“Accelerated C++”*, 6.ed, Addison-Wesley, 2003
- S. Lippman *“C++Primer”*, Addison-Wesley, 2013
- M. Bancila *“Modern C++ Programming Cookbook”*, 2.ed, 2020
- D. Kusswurm *“Practical C++ STL Programming”*, Apress, 2024
- R. Schneeweiss *“Moderne C++ Programmierung”*. 2.ed, Springer, 2012.
- S. Dmitrović *“Modern C++ for Absolute Beginners”* 2.ed, 2023
- I. Horton, P. Van Weert *“Beginning C++23”* 7.ed, 2023

Hello World

- seit C++23

```cpp
// file: hello23.cpp

#include <print>

int main() {
  std::println("Hello, World!");
}
```

```
g++ -std=c++23 -o hello23 hello23.cpp
./hello23
```

```
Hello, World!
```

- vor C++23

```cpp
// file: hello.cpp

#include <iostream>

int main() {
  std::cout << "Hello, World!\n";
}
```

```
g++ -o hello hello.cpp
./hello
```

```
Hello, World!
```

Hello World Auffälligkeiten

- Header ohne Endung `*.h`
  - C-Header entweder z.B. `<stdio.h>` oder `<cstdio>`
- Elemente aus der Standardbibliothek liegen im Namensraum (*namespace*) `std`
  - Aussprache wie engl. *stood* `[/stʊd/]`
- Moderne Ausgabe mit `std::print()`
- frühere Ausgabe mit Strömen (*streams*) wie `std::cout` (Standardausgabe)
  - Überladen von Operatoren, z.B. Ausgabeoperator (Bit-Shift) `operator <<`
- viele Ähnlichkeiten zu C
  - Verwendung von Bezeichnern (*identifier*)
  - `// Kommentare` und `/* Kommentare */`
  - `#include` von Deklarationen aus Headern
  - `main()`-Funktion
  - Verwendung von Zeichenketten `const char *`